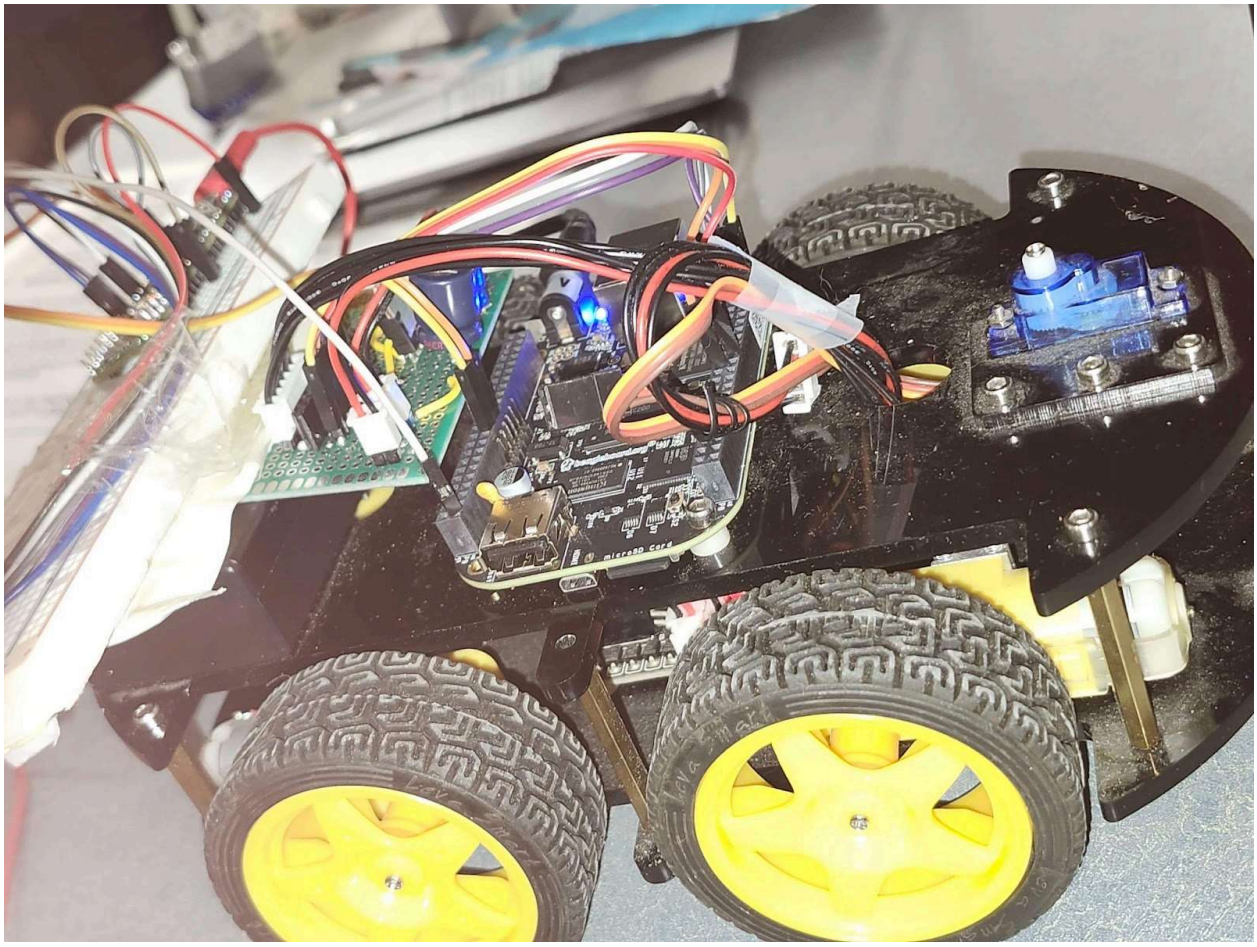
Super car

Bristine Teneng (bmt756), Devam Punitbhai Patel (dns682), Ryan Schaffer (rys686), Tianze Kuang (wde364), Uchechukwu Udenze (gvs690), Vlad Akulov (onc736), Wahhaj Javed (muj975)

1. MVP

SUPER COOL CAR



For the minimum viable product, we used the beaglebone to drive an ELEGOO Smart Robot Car. The car drives wirelessly by receiving input from a joystick sent over an nRF24 transceiver. There is a matching transceiver connected to the beaglebone and the beaglebone handles the incoming signal and drives the motors accordingly.

2. Build

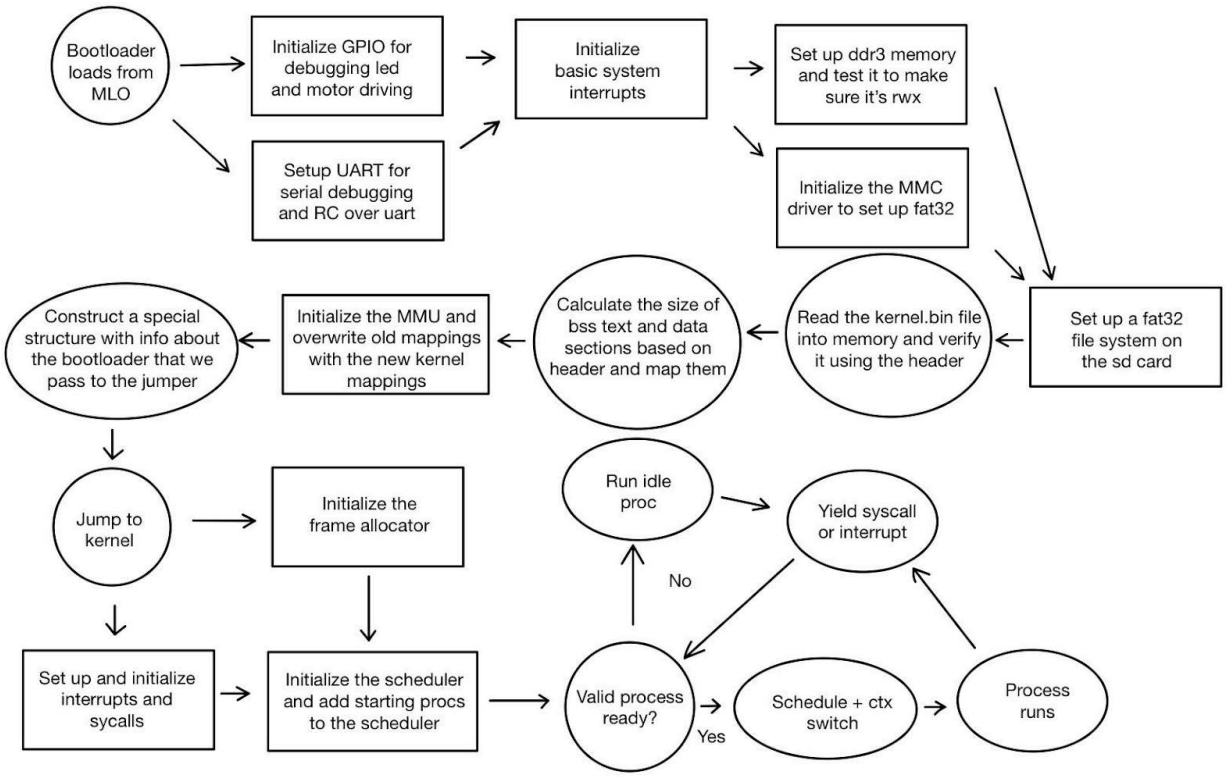
Must be on linux with the Arm GNU Toolchain installed. Follow these steps to compile and upload the code

- connect sd card
- locate block dev name

- sudo make flash DEV=/path/to/sd_device

Hold down S2 button before powering on to boot from the micro sd card

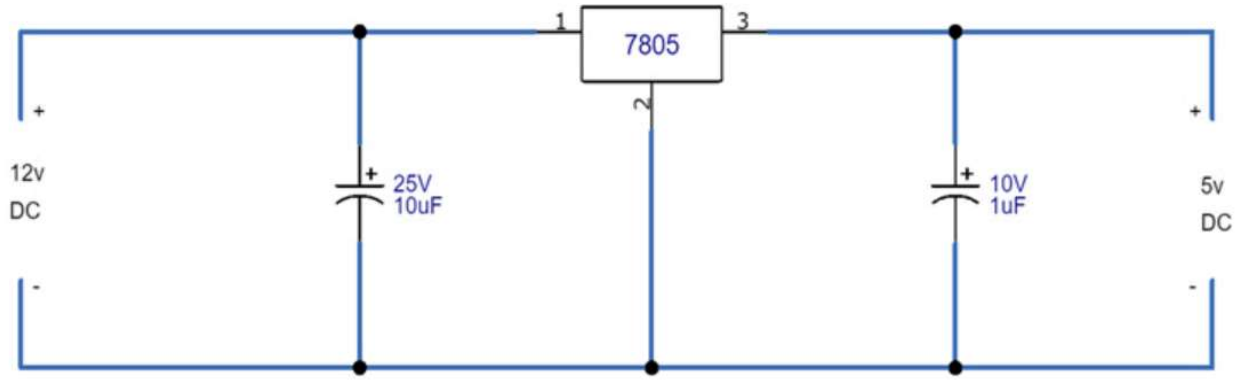
3. Design decisions:



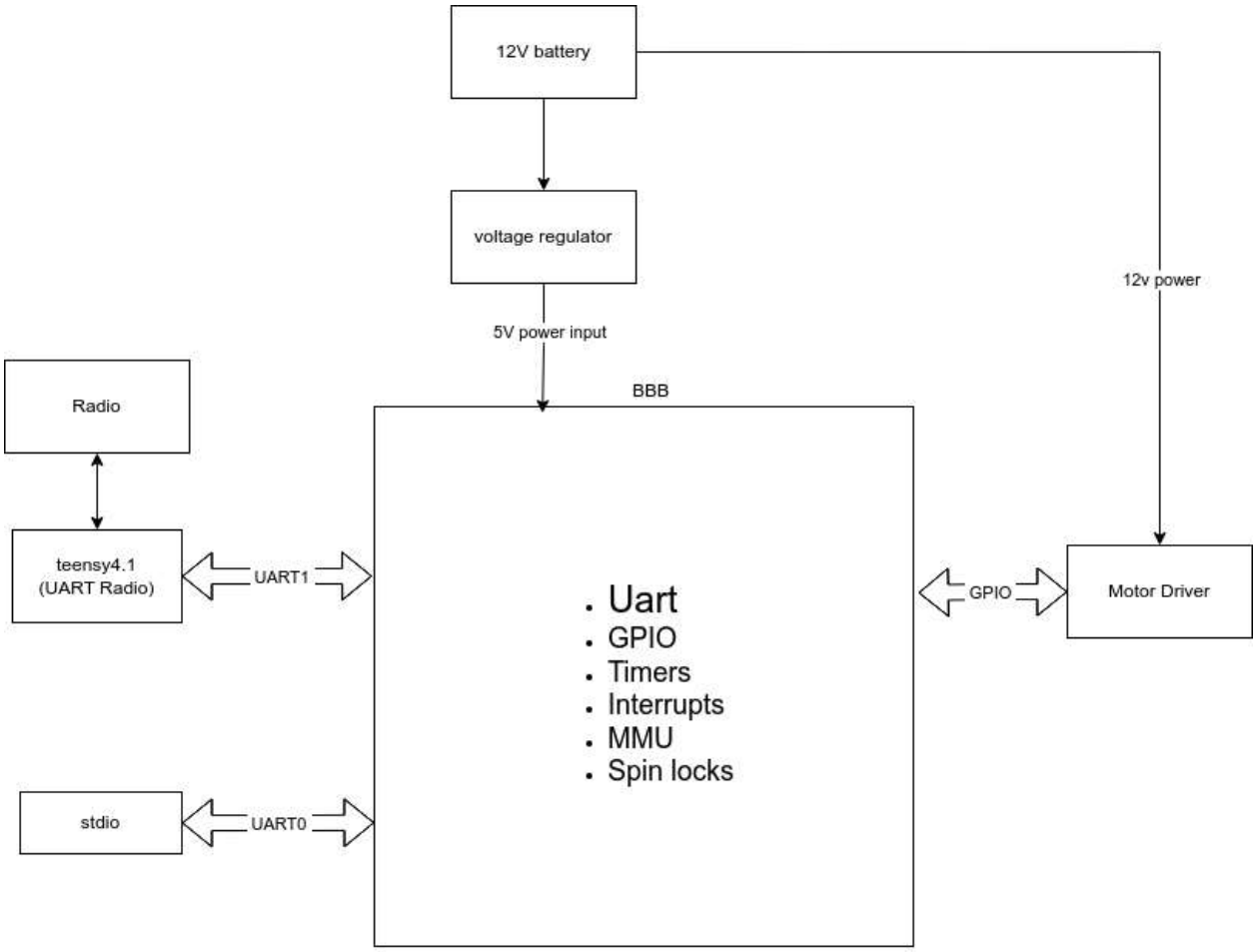
3.1. The car

We based our hardware design around an ELEGOO Smart Car kit, which already included most of the components we needed for our MVP —including the chassis, motors, wheels, motor driver, and power system. To adapt the kit for our project, we replaced the original microcontroller (an Arduino Mega) with the BeagleBone Black, allowing us to run our own custom OS and scheduler. The car is powered by a 12V battery, which is suitable for driving the motors, but not directly compatible with the BeagleBone Black, which requires a regulated 5V input. To resolve this, we designed and built a voltage regulator circuit that steps the 12V supply down to 5V, powering the BeagleBone while still delivering the full 12V to the motors. We connected the motor control signals from the motor driver board to a set of GPIO pins on the BeagleBone. These pins are configured as outputs and are set or cleared by the software to control motor direction and enable signals, giving our system full control over the car's movement.

Voltage regulator circuit:



Overall hardware architecture:



3.2. Bootloader

Our bootloader for this operating systems project had a couple of things it needed to accomplish. First of all we have to have it loaded in at all. We chose to use an MLO because it was easier since we found an example online that showed how to coerce our bootloader to be loaded in from the sd card. The Second thing that our bootloader was responsible for was initialising the hardware we would be using in our car

which would be the following as a bare minimum. It had to initialise GPIO, UART, DRAM, and MMC each of these were crucial toward the basic functionality of our code.

We chose to use GPIO to drive our motors as it seemed like the most sensible and simple option where two pins would connect to one motor where a 1/0 would mean forward and a 0/1 would be backward using a tank turning to turn the car. Uart would be just as crucial as it is the #1 easiest way to debug our code outside of a virtual environment as well as effective and simple for transmitting our RC signal to the vehicle itself through a module.

Some hardware less crucial for hardware operation but necessary for our operating system to function at a basic level was the DRAM and MMC. The DRAM to load our operating system in any capacity as the onboard SRAM would be far too small to carry both the bootloader and the operating system itself. The other crucial bit we had spin up was the MMC reader to read the contents of our sd card that contained the os we had to load into our DRAM.

Due to the fact that we spun up so many modules some of which like DRAM had outside controllers and modules we had to communicate with that may not have been instantly apparent when they broke or were not working properly so we chose to initialise interrupts to have our processor interrupt and stop execution if something had gone wrong so we knew where the problem was and could isolate and fix it.

Our initialisations all get abstracted away by their respective init functions to keep the main bootloader.c file clean and free of most low level logic. However not all problems could be identified and solved via interrupts and some problems silently fail or get misconfigured all the same. It is for this reason that some of our bootloader initialisations have two parts to them. The first part being the initialisation portion that actually sets up the data structures enables the hardware, the pll's and all associated functions. The second part of our init process is a test that confirms that the component is working in the case of UART that could be as simple as a uart_puts and a visual confirmation then a uart_getc and then a uart_putc on that character to confirm all function are working or it could be complex like reading and writing patterns to the dram to make sure there are no dram faults and that both the reads and writes are being performed successfully.

To enable virtual memory and isolate kernel space we initialize our MMU and create a simple one to one mapping and enable rwx on it for the kernel to make it easier to use.

After having initialised all of our modules the next responsibility of our bootloader was to find and load our operating system from the SD card into DRAM since we initialised the MMC a simple sd card reader on top of it can read the sd card that has our MLO and find a kernel.bin that can be read directly into memory.

Before actually performing the jump to the kernel we verify the image is correct using a magic number in the header and that gives us the confidence that what we loaded was the expected kernel build. Once we validate the kernel we parse section information from the header and zero out the .bss region. Each section of the kernel gets mapped with the MMU and logging confirms correct mappings.

Just before we perform a jump to kernel space to pass some metadata to our kernel we construct a small boot info structure to pass to the kernel and fill out some key info like mapped section count and page table location. Finally our bootloader jumps to the kernels entry point handing execution over to the kernel.

3.3. Uart:

The BeagleBone Black has six UART ports, out of which we use two in our MVP: UART0 and UART1. UART0 is used for standard input and output, primarily for debugging via a serial console. This port is initialized early in the boot process so that logs can be printed from the start of execution.

UART1 is used to receive joystick input sent wirelessly from the controller. This input is forwarded to the BeagleBone over a UART connection by a secondary microcontroller (Teensy 4.1), which handles the actual RF communication (see Section 2.9 for more details).

For both UART ports, we only enable Receive Holding Register (Rx) interrupts. These interrupts trigger when new data is available in the UART buffer, allowing us to process incoming data efficiently without polling. All other types of UART interrupts — such as transmitter holding empty, line status, or modem status — are disabled, as they are not necessary for our current use case.

While the UART hardware on the BeagleBone supports more advanced functionality such as DMA transfers and hardware flow control, we do not enable these features. This is a conscious design choice to reduce development time and complexity, as our MVP does not require high-throughput or low-latency serial communication.

3.4. Interrupts:

The BeagleBone Black exposes up to 128 interrupt lines, each associated with a specific peripheral or system event. To fully support this capability, we designed a general-purpose interrupt system that can handle any valid interrupt source dynamically. The goal was to build a foundation that would scale well with the rest of the system as we added more device drivers and expanded functionality.

Our first design decision was to set up the interrupt controller and vector table at system initialization. We configured all interrupt lines to initially point to a default handler. This served both as a debugging aid and a safeguard during development by ensuring that spurious or unregistered interrupts wouldn't lead to undefined behavior.

We wrote functionality to allow interrupt handlers to be registered dynamically, giving each peripheral or module full control over how it responds to events. At the same time, we enabled precise interrupt masking and unmasking, allowing any interrupt line to be enabled or disabled as needed. This was particularly important for debugging and for ensuring that only relevant interrupts were processed during different phases of the system's operation.

We also provided functionality to enable or disable interrupts globally at the CPU level. This is crucial for entering critical sections or controlling interrupt-driven behavior during context switching and system calls.

3.5. Timers

The BeagleBone Black contains eight hardware timers: Timer0 through Timer7. Each timer has its own set of features and intended use cases. Timer0 supports low-power wakeup events, and Timer1 is capable of high-precision 1ms timing using a different clock source. These two timers are slightly more complex to configure and require different initialization routines.
To simplify development, we chose not to support Timer0 or Timer1. Instead, we support and use only Timers 2 through 7, which share a consistent configuration and usage model. This design decision was made to reduce code complexity and to accelerate development by avoiding the need for special-case handling of Timer0 and Timer1.
Timers 2 through 7 can be configured to use either the system clock (which can be adjusted between various frequencies) or a 32.768 kHz clock source. We decided to use the 32kHz clock, as it is accurate enough for our timing needs (e.g., scheduling, task switching, timeouts), and will remain unaffected if we change the main system clock in the future. Using the 32kHz clock ensures that timer behavior remains stable and predictable regardless of the BeagleBone's CPU frequency.

3.6. MMU:

The MMU in our design was meant to provide memory protection and enable a simple, section-based memory model. We planned to use 1MB sections exclusively, making use of the ARM L1 section entries to keep things straightforward. Our approach centered around having a frame allocator that managed these 1MB-aligned physical memory blocks. The allocator was responsible for tracking which frames were free and handing them out when new memory was needed.

We implemented a basic allocator that initializes on boot by walking through physical memory and linking together any unused 1MB blocks. These blocks could then be handed out one at a time through the alloc_frame() function. This kept the frame allocation lightweight and deterministic, avoiding fragmentation.

To support mapping, we added a command-style API that allowed any 1MB-aligned virtual address to be mapped to any 1MB-aligned physical address with specified access flags. This is done through the MMU_map_section() function, which inserts the appropriate section descriptor into the L1 page table. We also provided helper functions for flushing TLBs, enabling and disabling caches, and setting domain access levels, which gave us more control over how memory was accessed and cached.

Hardware sections were pre-mapped during MMU initialization using this same mechanism, and all mappings were configured for read-write access with optional caching. Debugging tools like log_l1_pte() and log_vaddr_mappings() made it easy to verify our memory layout at runtime and ensured that our page tables were being built correctly.

Although not all features were used to their fullest extent, we succeeded in building a minimal, consistent MMU setup that supports memory protection and deterministic memory layout using clean 1MB blocks.

3.7. Processes:

The design philosophy for processes in our project was to make them simple but easy to modify later. We planned to have as many different processes as there are free pages in memory which would be 512 on bootup minus however many the kernel and IPC use. We decided the easiest way to account for the mmu was for a process to take up exactly one page of the MMU, so 1MB, and tile until we hit the end of our RAM. This was not fully realised in our final product as we could not get the MMU to behave so processes just directly point to code pre-loaded in the kernel and the maximum number of processes is limited by a MAX_PROCESSES variable set to 32.

We planned to achieve full process isolation so we made each page have its own code, stack and heap section. The plan was for the size of each section to be fixed at an arbitrary size to simplify memory management for OS development. In reality since the code is just pointing to a place in the kernel we don't have a place for stacks outside of the kernel initialising an individual per process stack in the kernel.

We planned that when a process is copied into memory it is then assigned a PCB that contains the necessary information for the process to operate including a pid, creation time, priority, registers, sp, lr, cpsr, pc, segment base, state, and a timestamp of when it was last ran. We wanted to go super minimal for the process control block to simplify context switching and process management needing to only save and restore registers along with keeping track of when it switched out of for the scheduler and state to keep track of running, blocked and ready processes. We achieved most of these things. We have all of the process descriptors in order to context switch. We didn't get the chance to add any timestamps to the process but we have a pid and scheduler name for debugging.

In our paged design we want to have a segment base store the address of the allocated page in order for us to know where we need to clean out of one the process terminated or if we need to move it in the future. Processes would have been dynamically loaded from disk at runtime when the kernel or scheduler decided to launch a new process. Dynamically loading the processes based on name would have been a non conventional way of doing it but would have allowed us to quickly develop and modify our code to add a new process and load it in where it needed to be loaded in. Instead our processes currently as stated above aren't page bound and in consequence are simpler but more limited in scope.

We planned to have the OS read the process binary from the FAT32 file system and allocate a dedicated 1MB page-aligned memory region for it. Each binary would have included a small header with a unique signature (0x600D1DEA) provided by the linker, which the loader checked to ensure the binary is valid. This signature would have also helped identify the entry point for execution and enable safer process management. The way the processes would have been loaded and set up using the MMU to obtain a page made them completely isolated from the rest of the kernel, say for the shared ipc. Again we did not reach this far due to some difficulties with the MMU.

3.8. Scheduler:

To schedule our processes, we wanted to opt for an interrupt-based scheduler that preempted the current running process, stored its registers and necessary info into its PCB and scheduled the next process. We thought this approach would have been best for a real time vehicle in case a main loop design may be constantly polling and if a different process had to run to say update the lcd or something that doesn't run very often or even for long. We thought to preempt the main loop to execute that code and then that code could yield back or be prempted to receive radio commands. In reality our scheduler is a yield based scheduler a process should terminate or call what is essentially a yield function within the code to allow the process to schedule a new process or just hand it back to the current process

The scheduler maintains a fixed-size process table to keep track of all runnable processes that are dynamically added through the scheduler add process command. Due to the planned interrupt architecture we wanted the scheduler to be called after every single system interrupt to account for ipc blocking the current process, or other conditions like the current process being killed which would have needed a new process to be scheduled. Since we don't have interrupts we have to rely on the processes being nice and frequently yielding back to the scheduler and utilize none of our blocking ipc if we are not sure we will have them be unblocked.

In our plan for an interrupt based system the scheduler should have first checked if the current process is still alive and not blocked; if it wasn't it switched back to user mode to the process. Our planned scheduler would have run every interrupt and if a scheduler interrupt happened it would have set a flag that made the scheduler pick a new process to run. Unfortunately due to our time constraints because our kernel does not actually have interrupt scheduling we don't check anything about the process we just run the algorithm and put in the next process.

The scheduler itself is a pluggable modular design which allows us to easily change algorithms if we feel the need to choose some kind of rtos algorithm. The default is round robin but we can switch the scheduling algorithm at any time by changing the schedule next pointer.

When initialising the scheduler an idle process is found and loaded in to be run when no process is available. The idle process calls the scheduler to schedule the next process so if a process appears it runs instead. The current process is tracked by a global variable available to all kernel code to modify in case we need to change something in the process pcb such as its status to make development easier.

3.9. GPIO:

The beaglebone black has 4 GPIO ports. We only initialize GPIO1 as it enables us to control the debug LEDs (GPIO1_21 to GPIO1_24) and has enough pins on the exposed header for our use case. We use some of those pins (GPIO1_2 to GPIO1_7) to control the motor driver

3.10. Joystick input (over uart RF):

To enable wireless control of the car, we use two nRF24L01+ transceivers, one on the handheld controller and one on the car. These modules communicate over a 2.4 GHz RF link and require an SPI interface for communication with a host microcontroller.

Although the BeagleBone Black has multiple SPI ports available, we decided not to interface directly with the nRF24 transceiver over SPI from the BeagleBone. Instead, we added a separate Teensy 4.1 microcontroller on the car side to handle the SPI communication and RF protocol. The Teensy uses an open-source nRF24 library to receive joystick input and then outputs the received data over a UART connection to the BeagleBone.
This design choice was made to reduce development complexity and save time. Directly interfacing with the transceiver would have required implementing a full SPI driver and custom RF communication routines on the BeagleBone. Given the limited development time and the complexity of reliable RF communication, we chose to treat the Teensy as a standalone peripheral. It acts similarly to how a Bluetooth dongle works for a PC —abstracting away the wireless communication and presenting a simple serial interface.

3.11. Spinlock:

This is a hardware spinlock implementation. Since at the time of writing spinlock, we did not yet have a kernel or processes, the bare minimum was implemented before moving to anything more advanced. There were also no applications for our OS that really warrants a complex spinlock or counting semaphore since our goal was to drive a car. This is a very simple try_lock spinlock, non blocking spinlocks. It gets the job done and since there was no application for it, we left it there as a nice to have.

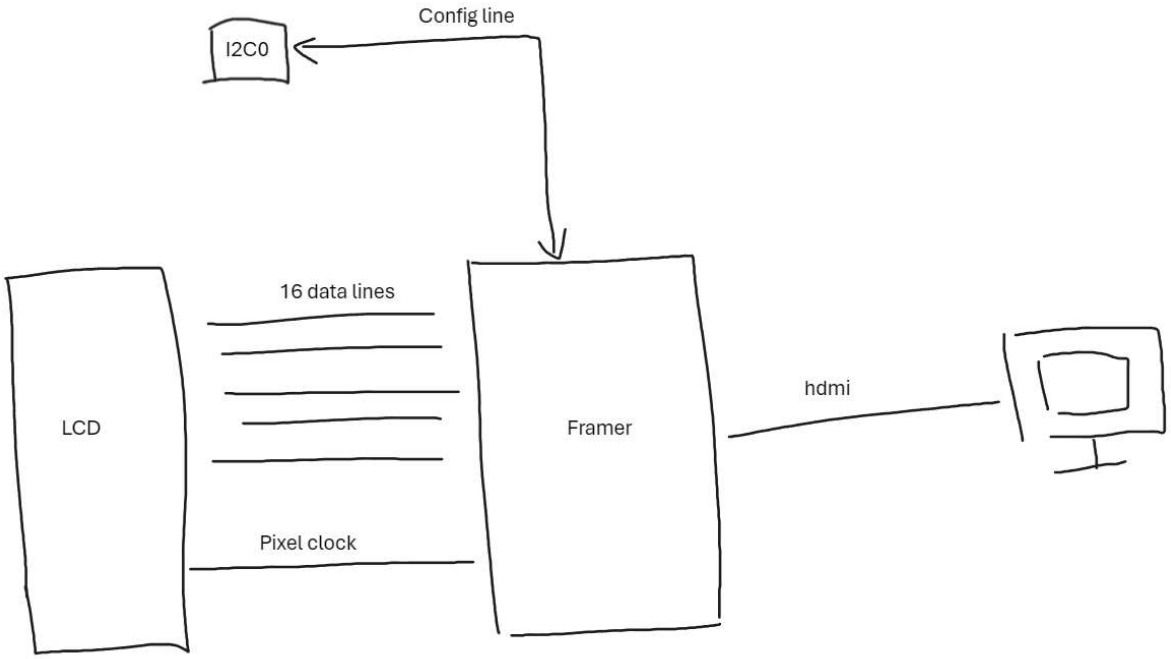
3.12. MailBox:

Similar situation to the spinlock, there was no reason for our OS to pass messages either, so it was a nice to have. The design was very simple since a lot of FIFO details were abstracted away in the hardware. All the real trouble arises from finding the correct clocks to enable and in what order to write the registers, which was not all that difficult. All 8 Mailboxes are configured once the user calls init(). Each mailbox can hold 4 messages, each message is 32 bit. In total you can have 8 * 4 = 32 messages at once. It would be almost trivial to convert the messages into pointers into memory, this way we can enable arbitrary message length. Issue being that there were no processes and no way to malloc when we implemented the mailbox. As a user, you can use this base to write your own mailbox abstraction. The current mailbox internally polls IRQ RAW status registers before read or write operations, none of the functions are blocking, it will either fail or succeed. No real need for a mailbox that calls the interrupt handler to drive a car.

3.13. I2C:

The I2C was developed for the sole purpose of initializing the hdmi framer tda19988 on board the BBB. We determined that I2C was not important to our MVP so it was designed with simplicity in mind. There were no interrupts set up for I2C, all interrupts are managed manually and every read/write operation will block until the expected interrupts happen. IRQ are polled from the IRQRAW register. Although this is inefficient, I2C only had to be called once at initialization. All three I2C ports (I2C0, I2C1, I2C2) have been configured to work but they must be individually configured at runtime using I2C_init(). Again since it is only used for the sole purpose of configuring the hdmi framer (tda19988), this interface is

limiting and might need to be modified in the future if it is used for more generic communications. A diagram is provided below to demonstrate the relationship.



The I2C was difficult in that the I2C protocol was not implemented for you so I had to write it myself. This involved first figuring out that I had to implement the I2C protocol which was not obvious from the documentation, then learning about how the protocol works. Which packets and in what order should they be sent in. Who is a master and who is a slave? In hindsight I should have asked my group members about it since they are computer engineers, but I managed to figure it out with a lot of struggle. One particular pain point was that during send and receive, there were arbitrary delays that I had to put in order for the module to function properly, again not documented anywhere. Since I chose to keep it simple and not use interrupts, when I first send the slave address register, I must wait a few seconds for the i2c module to send the address over the bus before moving on, or else the entire module comes to a halt. The only reason that I was able to diagnose this issue is because I had a fully working I2C, once I started removing print statements, it began to behave erratically. After several hours of struggling, I was able to finally determine the cause.

I have designed I2C_init() to enable all the main L3/L4 clocks. This way almost all clocks that are needed for the board are together in one place. Additionally, now I2C can be called anywhere in the code and will not have to worry about missing clock initializations. For our use case, since the framer depends on I2C and there would be no output without the framer, this saves a lot of clock configuration for those sections. This was tested in conjecture with the hdmi framer. It was able to return a valid identification value when I called hdmi_framer_probe(). It prints tda19988 which is a very specific non zero version number. This value was consistently being returned back from the probe and printed.

3.14. Hdmi framer tda19988:

This module is one of the most problematic areas of this implementation. The official BBB technical reference says that to enable hdmi, this framer must be enabled and configured through I2C. From the description, it appears to be another module that needs to be configured. However after many days of struggling, it turns out that the documentation for the framer is locked under an NDA and is also no longer supported by nxp. Since there was no documentation, the only solution is to copy existing open source projects. I chose uboot's tda19988 implementation. There are many different implementations from different open source projects. I know that uboot works with BBB because that is what is used to load the linux kernel and the source code was also one of the simpler ones I saw.
The second battle is to find a way to integrate the tda19988.c implementation into our existing code base. As to be expected, uboot has entirely different structures than our kernel which means that many functionality and implementation details were different. Structures, enums, #define definitions etc are all missing and I had to manually look through each one of them and either copy it from uboot, find a way to incorporate / write my own, or delete them. Within the current tda19988.c file inside our kernel, all of the deleted code from uboot is commented out because I wanted to be explicit about my modifications and make it easier for other people to see how it differed from the original file. Uboot functions such as i2c_reg_read / i2c_reg_write had to be adapted to function properly. I had to copy the displaying_timing structure over and write a simple wrapper to preconfigure some timing information since we have no structures to determine the resolution of the monitor at runtime. There are also miscellaneous functionalities that I turned off because they were calling other udev functionalities which are

- Not important to configuring hdmi
- Require much more effort to port than just simply deleting them

Structure of the framer :

- An hdmi framer
- A on board cec chip

Due to a lack of documentation, it is not clear what the intended separation of tasks between these two modules onboard tda19988 is used for. However, cec appears to be a common and standard implementation that enables custom user configurations and aids the hdmi framer in providing hdmi packets. Without first enabling the cec chip through I2C, no subsequent I2C communication can happen with the framer at all.

Monitor configuration :

The display_timing structure was difficult because apparently every display have their own preferred timing info. Things like front and back porch values, vsync and hsync values are different between monitors. There are industry standard values, so I have configured them to those values, but since I do not have any documentation for the tda19988, I cannot say for sure which porch values are correct and which are not. I did cross reference with BBB's kernel's device tree configurations just to make sure, however I was not able to get an image to display. I will discuss this more in the LCD controller section.

After a lot of struggle, I was able to ping the framer to return back some identification information through the I2C bus, this way, I was able to claim that the I2C implementation indeed works and the framer also worked.

3.15. LCD driver:

The final chapter to the HDMI saga is that in order to have any hdmi output, the LCD driver must send pixel clock and data lines to the framer. Before the LCD controller, it was all about debugging the I2C communication, however I now had to teach myself basic HDMI communications.
The basics of HDMI signal goes as following :

- An image have M horizontal lines and N vertical lines -> M * N -> width * height
- For synchronization reasons, there are also non visible pixels called porches before and after visible pixels
    - Display ←– front_porch, actual_pixels, back_porch ←– LCD controller
    - And there is also sync bytes mixed in for h/v sync
- To derive a pixel clock, it is
    - Pixel Clock=(Horizontal Total) × (Vertical Total) × Refresh Rate
    - Each pixel clock is 1 pixel

The LCD is hard coded to output :

- 1280 x 720 @ 60hz
- 24bbp image data.
- hfront_porch = 110, hback_porch = 220, hsync = 40, vfront_porch = 20, vback_porch = 5, vsync_len = 5.

These values are also hard coded into the hdmi framer.

Using these values :

- Horizontal total = 1280 + 110 + 220 + 40 = 1650
- Vertical total = 720 + 20 + 5 + 5 = 750
- Pixel Clock = 1650 * 750 * 60 = 74.25 mhz

Where

Pixel Clock = LCD clock / Div value

Now the challenge is that we have two LCD clocks to choose from :

- DISP PLL CLKOUTM2 = 24 mhz (allegedly from multiple sources, boot pin dependent)
- CM_PER_PLL_CLKOUT_M2 = 192 mhz
- CM_PER_PLL_CLKOUT_M5 = 250 mhz

There are now two requirements, the LCD Clock must be less or equal to 200 mhz, but the pixel clock must be as close to the compute pixel clock as possible.

To obtain 74.25 mhz is now non trivial. Since those clock values are all even numbers, I need to modify the clock multiplier and divider with odd values since all multipliers must also be integers. I can only choose the DISP_PLL_CLOKCOUTM2 in this case because this is the only clock that I can change its multipliers before passing to the LCD controller.

Through experimentations, by selecting the M2 clock :

$$192 * 41 / 106 = 74.264 \text{ mhz}$$

Due to implementation details, am335x requires that the input LCD clock be divided by value larger than 2 to obtain a valid pixel clock which we can use to feed the framer. Therefore :

$$192 * 41 / 106 = ((24 * 8) * 41 / 53) / 2 = 74.264 \text{ mhz}$$

Now this is not the exact value that we are looking for but is the best we can get.

$$192 * 41 / 53 = 148.53 \text{ mhz}$$

Thus this is theoretically a valid clock signal configuration.

To arrive at this value was a lot of pain. As mentioned in the hdmi framer section, there was no documentation, so it was all guesses. These exact numbers are referenced off of BBB's device tree configurations and they claim it works since BBB does have hdmi output.

I have also tried many different resolutions like 1080p, 1024 x 768 @ 60 hz, 1024 x 768 @ 50 hz, and many others. But due to clock synchronization issues, I decided to change them.

The LIDD controller is for driving actual LCD panels / character displays, handling clocks and other physical display configurations so it was disabled.

To handle raw 24bpp (RGB) data, the LCD is configured to TFT active matrix mode, 24 bit unpacked. Only one frame buffer (frame buffer 0) is enabled so double buffering is currently not supported. Double buffering does have performance benefits, but displaying a static image, it only brings complexity without all of the benefits. The reasons that the Frame buffer must be allocated in DRAM is due to the design of the internal LCD dma engine. It is only designed to read from the DRAM address space and it takes in a 30 bit pointer, which limits it to 1GB address space. Additionally the frame buffer must be in this format : {[0] = 0x400, [1-32] = 0, [33 - n] = PixelRGB}. This is what the LCD controller expects from the frame buffer. I could abstract it so that the user only writes to the [33-n] portion of the frame buffer, however since this is a very bare bones implementation that only aims to display a picture as a proof of concept, it is currently not implemented.

After all the work, the beagle bone black is able to turn on any arbitrary hdmi display for around 15 - 20 seconds before going back to sleep. However, I was not able to get an image to display.

Digging deeper into the IRQ status of the LCD controller, it reports

- Irq status : 1 and 2 and 8

Where for the first test where I provided an invalid framebuffer layout
These corresponds to IRQ

- Raster Mode Frame Done Interrupt (1)
- Frame Synchronization Lost Raw Interrupt Status and Set Read indicates raw status (2)
- DMA End-of-Frame 0 Raw Interrupt Status and Set Read indicates raw status (8)

As we can see 1 and 8 indicate that it detected an invalid frame buffer (1), and so the DMA engine ignored it (8). However, the issue lies with IRQ (2). Frame synchronization Lost means that the pixel clock provided has a mismatch with the pre-programmed data dimensions inside the framer. This explains

why the display turns on, as it was able to receive proper clock signals, however, since no data was ever loaded and transferred, it failed to synchronize with the framer since the agreed upon data format was never delivered.

Now with the proper configured frame buffer layout it reports :

- Irq status : 2 and 5

These corresponds to IRQ

- Frame Synchronization Lost Raw Interrupt Status and Set Read indicates raw status (2)
- DMA FIFO Underflow Raw Interrupt Status and Set LCD dithering logic not supplying data to FIFO at a sufficient rate, FIFO has completely emptied and data pin driver logic has attempted to take added data from FIFO. (5)

This is interesting since from these IRQ reports, it is clear that there is some issue with the DMA engine. Specifically, it sends data faster than it can read from DRAM. Thus it suggests a potential clock issue between the DMA and DRAM. My group member confirmed my suspicion as the DRAM is initialized to run at 118.5 mhz. Now, looking back at the calculation, the LCD controller is configured to run at 148.53 mhz and the pixel clock is 74.26 mhz. This suggests that the DMA is running on 148.53 mhz which is faster than what it can read from DRAM. However, since the pixel clock is much slower than DRAM, it does not make sense for IRQ 5 to happen.

The Frame synchronization Lost IRQ is easy to explain since in the first test, no data was ever transferred, so no synchronization was possible, while in the second test, the DMA was not able to provide enough data at the right time for synchronization to properly occur. From the above two tests, I am convinced that this is a clock misconfiguration issue.

Work is currently being done in tweaking one or all of the DRAM, LCD and Pixel clock multiplier values to see which configuration would improve the current conditions. Currently, progress has been slow, no matter which multiplier values I tweak, either the display turns on and I have a synchronization issue, or the clocks are so far off that the display simply does not even acknowledge and turn on. Although hdmi will not be able to make it to the final deliverable in time, I will work on it until it functions properly.

3.16. MMC driver:

We load our kernel over the SD/MMC interface because it supports a filesystem, it is fast and automatic. Loading the kernel over other alternatives like UART would require manually sending the image over a serial link which is slow and doesn't provide a filesystem support. Our MMC driver code is very simple and straightforward. Because we boot from the SD card, most of the initialization is performed automatically for us and we went through a lot of troubles before realizing this. Initially, we tried to follow the manual to initialize the controller and experimented with a lot of different settings like changing the clock frequency, bus voltages and more but our command 8 was not responding and we kept getting timeout errors before we switched to this simplified initialization code where we just make sure clocks are on, set bus voltage, change a few other configs like preventing the controller from entering low power mode and preventing clocks from turning off, and it started working. To keep things simple, we poll the SD_STAT register instead of using MMC interrupts. Our read sector implementation reads a

single 512 bytes block and we haven't added multi block reads, DMA support, or sector writes as they were not required for our MVP.

3.17. Fat32 parser:

We chose to implement a Fat32 parser rather than hard coding sector offsets because it lets us refer to kernel.bin by name and not by numeric addresses which makes it easy for us to swap or modify image if we want without having to recalculate offsets. We only planned on using this FAT32 parser for loading the kernel so, our FAT32 parser only implements mount, open and read. We haven't added implementations for write, delete, or file system modifications because they were not in our MVP. To avoid using libc we have our own simple strlen, strchr, toUpper, memcmp, strncpy, and memcpy which are just enough to parse paths and compare short filenames. Each sector is read and processed immediately. We had plans for adding caching but decided to leave it as it was nice to have. On any I/O or format error we simply panic as adding recovery was not necessary for our MVP.

4. API:

4.1. Interrupts:

void INTC_init(void)
Initializes the Interrupt Controller (INTC). This function performs the necessary low-level setup of the INTC, including resetting it, configuring clock modes, and setting the priority threshold. It also initializes the interrupt vector table with default handlers. This function must be called once during system startup before any interrupts can be used.
Parameters: void.
Returns: void.

bool INTC_register_irq(uint32_t int_num, void (*handler)(void))
Registers a user-defined interrupt handler function for a specific interrupt number. This function updates the interrupt vector table with the provided handler address. The registered handler will be executed when the corresponding interrupt is triggered and enabled.
Parameters:

- **uint32_t int_num**: The interrupt number to register the handler for. This number must be a valid interrupt number supported by the system (less than NUM_INTERRUPTS).
- **void (*handler)(void)**: A pointer to the interrupt handler function. This function should have no parameters and return void. It will be called by the system when the registered interrupt occurs.

Returns:

- **true** If the handler was successfully registered for the given interrupt number and **false** If the int_num is out of the valid range (greater than or equal to NUM_INTERRUPTS).

bool INTC_enable_irq(uint32_t int_num)
Enables a specific interrupt, allowing it to be processed by the CPU when triggered. This function clears the corresponding mask bit in the INTC's mask register. The interrupt must also be enabled at the CPU level for it to be fully active.

Parameters:

uint32_t int_num: The interrupt number to enable. This number must be a valid interrupt number supported by the system (less than NUM_INTERRUPTS).

Returns:

true if the interrupt was successfully enabled at the INTC level and **false** if the int_num is out of the valid range (greater than or equal to NUM_INTERRUPTS).

bool INTC_disable_irq(uint32_t int_num)

Disables a specific interrupt, preventing it from being processed by the CPU when triggered. This function sets the corresponding mask bit in the INTC's mask register. This will prevent the interrupt from reaching the CPU, even if it is enabled at the CPU level.

Parameters:

uint32_t int_num: The interrupt number to disable. This number must be a valid interrupt number supported by the system (less than NUM_INTERRUPTS).

Returns:

true if the interrupt was successfully disabled at the INTC level and **false** if the int_num is out of the valid range (greater than or equal to NUM_INTERRUPTS).

void CPU_irq_enable(void)

Enables the ARM CPU's general interrupt processing by clearing the I-bit (IRQ mask bit) in the CPSR (Current Program Status Register). This allows the CPU to respond to enabled IRQ interrupts from the INTC.

Parameters: void.

Returns: void.

void CPU_irq_disable(void)

Disables the ARM CPU's general interrupt processing by setting the I-bit (IRQ mask bit) in the CPSR. This prevents the CPU from responding to any IRQ interrupts from the INTC, even if they are enabled at the INTC level.

Parameters: void.

Returns: void.

void system_interrupt_init(void)

Initializes the entire interrupt system. This function performs the following crucial steps: sets up the vector table in memory at a predefined location (VECTOR_TABLE_DEST_ADDR), enables CPU interrupt processing, and initializes the Interrupt Controller (INTC). It is essential to call this function once during system startup to enable interrupt handling.

Parameters: void.

Returns: void.

4.2. Syscalls:

void setup_syscall_table(void)

Initializes the system call vector table. This function populates the `syscall_vector_table` array with the addresses of the implemented system call handler functions. It must be called once during system initialization to make system calls available.
Parameters: void.
Returns: void.

int handle_syscall(uint16_t syscall_num, ...)
Dispatches a system call based on the provided system call number. This function takes the system call number and a variable number of arguments, retrieves the corresponding handler function from the `syscall_vector_table`, and executes it. It also includes error checking for invalid system call numbers.
Parameters:

uint16_t syscall_num: The number identifying the system call to be executed. This number corresponds to an index in the `syscall_vector_table`.

...: A variable number of arguments that are passed to the invoked system call handler function.

Returns:

int: The return value of the executed system call handler function. Returns `-ENOSYSCALL` if the provided `syscall_num` is invalid or if no handler is registered for that number.

int sys_default(va_list args)
A default system call handler. This function serves as a placeholder or a default action for unhandled system call numbers. It currently performs no specific operation and returns 0. This is also called when a SYS_YIELD syscall is called as all syscalls call the scheduler after execution
Parameters:

va_list args: A variable argument list, which may or may not be used by this handler.

Returns: 0.

int sys_read(va_list args)
Handles the `SYS_READ` system call. This function is intended to read data from a specified file descriptor into a buffer. The current implementation sets a GPIO pin and returns 0 as we ran out of time to fully integrate it with a device tree table and a file system.
Parameters:

va_list args: A variable argument list containing the arguments for the `read` operation. The following parameters are expected:

- **uint8_t fd**: The file descriptor to read from.
- **char* buffer**: A pointer to the buffer where the read data will be stored.
- **uint32_t len:** The number of bytes to read.

Returns:

int 0 (current implementation). The actual return value should indicate the number of bytes read or an error code (negative).

int sys_write(va_list args)
Handles the `SYS_WRITE` system call. This function is intended to write data from a buffer to a specified file descriptor. The current implementation returns 0.
Parameters:

va_list args: A variable argument list containing the arguments for the `write` operation. The following parameters are expected

- **uint8_t fd**: The file descriptor to write to.
- **char* buffer**: A pointer to the buffer containing the data to be written.
- **uint32_t len**: The number of bytes to write.

Returns:

int 0 (current implementation). The actual return value should indicate the number of bytes written or an error code.

int syscall(uint16_t syscall_num, ...)

Triggers a software interrupt (SWI) to invoke a system call. This function takes the system call number and its arguments and initiates the execution of the corresponding system call handler in a privileged mode. The result of the system call is returned to the caller.

Parameters:

uint16_t syscall_num: The number of the system call to be invoked.

...: A variable number of arguments to be passed to the system call handler.

Returns:

int: The result returned by the executed system call handler.

4.3. Uart:

void uart_init(unsigned short uart_index, unsigned int baud_rate, unsigned short stop_bit_en, unsigned short num_stop_bits, unsigned short parity_en, unsigned short parity_type, unsigned short char_length); - Initializes the specified UART peripheral with the given configuration. uart_init takes the UART index (currently only 0 and 1 are supported), a desired baud rate, and various serial settings including stop bit enable, number of stop bits, parity enable, parity type (0 for even, 1 for odd), and character length (typically 8). It enables the necessary module and interface clocks for the selected UART, configures the pin multiplexing to route the UART signals correctly, and delegates to a lower-level port initialization function to configure the baud rate and line settings. It also sets up the UART receive interrupt and initializes the corresponding circular buffer for received data. This function must be called once for each UART before it can be used for sending or receiving data.

void uart_putc(char c); - Writes a single character to the UART0 transmit buffer. uart_putc blocks until the UART transmit holding register (THR) is empty, then writes the character to the register to send it. If the character is a newline ('\n'), it first sends a carriage return ('\r') to ensure proper formatting on terminals that expect both.

void uart_puts(const char* str); - Writes a null-terminated string to UART0 by sending each character one at a time using uart_putc. uart_puts iterates through the string and transmits each character, blocking as needed until the UART transmit buffer is ready. Newline characters are automatically followed by a carriage return due to the behavior of uart_putc.

char uart_getc(void); - Reads a single character from UART0. uart_getc blocks until data is available in the UART receive holding register (RHR), then reads and returns the received character.

void uart0_interrupt_init(void); - Initializes the interrupt handling for UART0. uart0_interrupt_init registers the UART0 interrupt handler (handle_uart0_irq) with the interrupt controller, sets its priority to 1 (as priority 0 is reserved for non-maskable interrupts), and enables the interrupt line.

void uart1_interrupt_init(void); - Initializes the interrupt handling for UART1. uart1_interrupt_init registers the UART1 interrupt handler (handle_uart1_irq) with the interrupt controller, assigns it a priority level of 1 (as level 0 is reserved for non-maskable interrupts), and enables the corresponding interrupt line.

void print_number(int32_t num, char base, bool is_signed); - Prints a 32-bit integer to UART0 in either base 10 (decimal) or base 16 (hex). Handles signed output for base 10 and always prints lowercase letters for hex. Converts the number to a string, reverses it, and prints it using uart_puts.

void uart_printf(const char* format, ...); - Formatted UART output function similar to printf. uart_printf takes a format string and a variable number of arguments, supporting a subset of format specifiers including %c, %s, %d, %u, and %x. It parses the format string, substitutes the arguments accordingly, and sends the result to UART0 using uart_putc and uart_puts.

void uart_vprintf(const char* fmt, va_list ap); - Core formatted output handler for UART0. uart_vprintf parses a format string and a va_list of arguments, supporting %d, %u, %x, %s, and %c. It dispatches to helper functions like print_number, uart_puts, and uart_putc to output each formatted argument.

unsigned int uart_readline(unsigned short uart_index, char* buffer, unsigned int buffer_size); - Reads a full line of input from the specified UART's receive buffer. uart_readline fills the provided buffer with characters from the UART buffer up to buffer_size - 1, stopping when a newline character ('\n') is read or the buffer is full. It null-terminates the string and returns the number of characters read. Returns 0 if the UART index is invalid or no complete line is available (lines == 0).

4.4. Process:

The following is a description of the syscalls we planned to implement in our operating system. Unfortunately due to the fact that we only have kernel level processes and we never developed other scheduling algorithms the only process functions we implemented are create process and add_to scheduler which add together to create spawn

int spawn(const char *name, bool mode); - Spawns a new process by looking up a compiled process with the given name in the fat32 partition. Returns the new process's PID or -1 if the name is invalid or the process could not be created. If you are in Kernel mode then you can provide a true to the mode argument to spawn the process as a kernel process otherwise spawn defaults to creating a user process. The name is the name of the process bin that was compiled and stored as a .bin in the process directory. Returns the pid of the spawned process so IPC can be initiated with the process.

void exit(); - Terminates the current process and frees its memory then passes execution to the scheduler to schedule the next process or idle.

int kill(int pid); - Forcibly terminates the process with the specific PID. Returns 0 on success or -1 if the PID is invalid. If executed in user mode this syscall can only kill other user mode processes. If called from user space on a kernel space process will always return a -1.

int getpid(); - Returns the PID of the calling process.

void yield(); - Calls the scheduler to yield execution to the next process in queue.

int set_priority(int pid, int priority); - Sets the scheduling priority of the specified process Returns a 0 on success, or a -1 if you were trying to change the priority of a process that has higher priority than you or try to set the priority higher than your current one. If called by a user process on a kernel process regardless of priority

int wait(int pid); - Blocks the current process until the specified process exists. Returns a 0 on success when the process is successfully terminated or a -1 if the pid does not exist.

4.5. SpinLock:

```
void spinlock_init();
/* 0 for success, 1 for failure */
int spinlock_try_aquire(enum SpinlockRegisters Reg);
void spinlock_release(enum SpinlockRegisters Reg);

/* 32 spin locks available */
enum SpinlockRegisters
{
        /* locks */
        Spinlock_LOCK_REG_0,
        Spinlock_LOCK_REG_1,
        …
        Spinlock_LOCK_REG_31
};
```

Usage :

1. spinlock_init();

2. Lock or unlock accordingly

4.6. Mailbox:

```
enum Mailboxes
{
        MAILBOX_MESSAGE_0,
        MAILBOX_MESSAGE_1,
        …
        MAILBOX_MESSAGE_7
};

enum MailboxStatus
{
        MAILBOX_SUCCESS,
        MAILBOX_INVALID,
        MAILBOX_FULL,
        MAILBOX_EMPTY
};

void mailbox_init();

/* appends a 32 bit msg to the mailbox.
 * only holds 4 msg at once per mailbox.
 * pointer to buffer must be valid
 */
enum MailboxStatus mailbox_try_send(enum Mailboxes, uint32_t Msg);
/* returns MAILBOX_EMPTY on failure */
enum MailboxStatus mailbox_try_retrieve(enum Mailboxes, uint32_t* Msg);

/* returns the amount of messages in the mailbox 0-4, values not in range is
 * wrong and indicates a corruption
 */
uint32_t mailbox_msg_available(enum Mailboxes);
```

Usage :

1. mailbox_init();
2. Try send or receive accordingly

4.7. GPIO:

void GPIO_init() - Initializes the GPIO. Right now, as we only used GPIO1 in our MVP, we only initialize GPIO1. We enable its clock and then disable its clock's gating so that it operates like a module.

void GpioSetPinMode(const enum GpioIOBase Gpio, const unsigned int PinMask, const enum GpioPinDirection Dir) - Its sets the mode of the particular Gpio bit which corresponds to a GPIO pin to either be an input or output pin. It does this by writing to a single bit in the GPIO Output Enable Register of the BeagleBone Black board.

void GPIO_set(unsigned int gpio_base, unsigned int pins) - Sets the value of a particular GPIO pin to 1. To do this though, instead of writing to just one value, we have to write to the entire GPIO Base address.

void GPIO_clear(unsigned int gpio_base, unsigned int pins) - Clears the value of a particular GPIO pin to 1. To do this, like set above, we write to the entire GPIO Base address.

void GPIO_get(unsigned int gpio_base, unsigned int pin) - This essentially returns the value that has been written to a GPIO pin. We do this by reading the read-only GPIO Data Input Register for that pin.

4.8. MMC driver:

void mmc_controller_init(void); - Initializes the MMC controller. Right now because we boot from sd card, most of the initialization is already done and we only do simple initialization like making sure clocks are on, set bus voltage, configure the controller to prevent it from entering low power mode or turn off clocks, reset cmd and dat lines, and set block size to 512 bytes.

int mmc_read_sector(unsigned int sector, uint8_t buffer[512]); - Reads the physical 'sector' and copies its bytes into the buffer. This function is used by the fat32 parser to load our kernel image into memory. It returns 0 on success and -1 and prints out the error message on error.

4.9. I2C:

```
/* sets it to pin mux = 0, pull up/down enable, pull up selected, receiver enabled, fast slew (400kbps), 7 bit slave address, own address = 1, master mode */
int I2C_init(enum I2C_Base);

/* returns bytes read, -1 for error */
int I2C_read(
        enum I2C_Base, int SlaveAddr, int SlaveReg, void* buffer, long len);
/* returns bytes sent */
int I2C_write(
        enum I2C_Base, int SlaveAddress, int SlaveReg, void* buffer, long len);

/* read and write 8 bit values */
```

```
/* returns value read */
int I2C_reg_read(enum I2C_Base, int SlaveAddr, int SlaveReg);

/* returns bytes written, -1 for error */
int I2C_reg_write(enum I2C_Base, int SlaveAddress, int SlaveReg, int val);

/* change current i2c configuration, called only after init */
void I2C_set_conf(enum I2C_Base, int I2C_ConfigMask);
```

And these are the configurations exposed to the user

```
enum I2C_Base { I2C0, I2C1, I2C2 };

enum I2C_Configuration
{
        I2C_StartConditionEnable,
        I2C_StopConditionEnable ,
        I2C_ReceiveMode,
        I2C_TransmitMode,
        I2C_MasterMode,
        I2C_SlaveMode,
        I2C_StartByteMode,
        I2C_FastMode,
        I2C_Enable,
        I2C_10bitAddress0,
        I2C_10bitAddress1,
        I2C_10bitAddress2,
        I2C_10bitAddress3
};
```

Usage :

1. I2C_init(<I2C of your choice>)
2. Read or write accordingly to your needs

Note :

- I2C_set_conf() requires deep i2c protocol knowledge, it is not advised to use it.
    - Should be called after init

4.10. MMU:

void MMU_init(void); - Initializes the MMU by clearing boot page tables, setting memory domains, and mapping hardware memory 1:1. Loads the L1 page table into TTBR0 and flushes the TLB. Required before enabling the MMU; assumes bootloader has set up initial memory regions.

void MMU_enable(void); - Enables the MMU by flushing the TLB and instruction cache, enabling data and instruction caches, and turning on the MMU. Assume translation tables are already set up.

void MMU_map_section(uint32_t *l1_base, uint32_t vaddr, uint32_t paddr, uint32_t flags); - Maps a 1MB section in the L1 page table from vaddr to paddr with the given flags. Both addresses must be 1MB aligned. Logs and error ig alignment is incorrect but returns anyway.

uint32_t read_ttbr0(void); - Returns the current base address of the L1 translation table. Useful for making sure the MMU state and page table setup.

uint32_t read_dacr(void); - Returns the current value of the Domain Access Control Register.

void i_cache_enable(void); - Enables the instruction cache.

void i_cache_disable(void); - Disables the instruction cache.

void d_cache_enable(void); - Enables the data cache.

void d_cache_disable(void); - Disables the data cache

void flush_tlb(void); - Flushes the entire TLB used after major MMU changes to ensure all address translations are refreshed

void flush_tlb_entry(uint32_t vaddr); - Flushes a single TLB henry by the provided virtual address. Used after remapping a specific section.

void flush_tlb_asid(uint32_t asid); - Flushes all TLB entries associated with a specific address space ID. Useful for when a multiprocess system needs to cull a process and its associated entries.

void flush_d_cache(void); - Flushes the entire instruction cache by issuing a CP15 cache maintenance operation.

void flush_i_cache(void); - Flushes the entire data cache using CP15.

void log_vaddr_mappings(uint32_t* vaddr); - Logs the L1 page table entry for the given virtual address by printing its corresponding descriptor. Useful for verifying MMU mappings during debugging

uint32_t alloc_frame(void); - Allocates a 1MB physical frame by popping the head of the free frame list. Returns the frame's physical address, or 0 if no frames are available.

4.11. HDMI:

This is inside the tda19988.h file :

Error codes :
#define ENXIO 6
#define ETIMEDOUT 110

Functions

```
/* used to probe the framer, prints to uart hdmi framer version : step 1, return 0 on success */
int hdmi_framer_probe();

/ * enable the framer : step 2, return 0 on success */
int hdmi_framer_enable();

/* query edid info : optional, see hdmi spec for return value */
int hdmi_framer_read_edid(uint8_t* buf, int buf_size);
```

Usage :

Prerequisite : **I2C0 must be initialized prior**

Enable following this order

1. hdmi_framer_probe();
2. hdmi_framer_enable();

Framer must be used in conjecture with I2C and the LCD controller. Detailed description is provided inside LCD controller API section

4.12. LCD controller:

```
typedef union
{
        struct
        {
                unsigned char Unused;
                unsigned char Red;
                unsigned char Green;
                unsigned char Blue;
        };

        uint32_t Data;
} PixelRGB;

/* frame buffer must be on ddr, returns immediately if FrameBuffer is not inside ddr
*/
void LCD_Init(const PixelRGB* FrameBuffer, uint32_t Size);

/* prints a list of current IRQ status to console through uart and then clears them */
void ProbeIntrCode();
```

To get hdmi output, follow this sequence of steps :

1. Build frame buffer as a contiguous array*
    a. See Note section for details
    b. This buffer must be allocated inside DRAM
2. I2C_init(I2C0)
    a. Framer is using i2c0 for BBB
3. hdmi_framer_probe()
4. hdmi_framer_enable()
5. LCD_Init(FrameBuffer, 1280* 720)

*Note the framer must be a contiguous array allocated inside DRAM that is in the format

{ [0] = 0x400 [1-31] = 0 [32 - n] = Pixel values }

In code it looks like

```
uint32_t* FrameBuffer = allocate(32 + sizeof(PixelRGB) * 1280 * 720);
memset(Buffer, 0, 32);
*Buffer  = 0x4000;
PixelRGB* Buffer = FrameBuffer + 32;

LCD_Init(FrameBuffer, 1280 * 720);
```

5. Testing

5.1. Interrupts

Most of the interrupt testing just involved using the interrupts in other parts of the system. For example UART input is done solely through interrupts and we use UART input to receive commands to drive the car. Initially, interrupts are tested using a series of print statements to indicate where and when interrupt happened, and those occurred at the expected places.

5.2. UART

Our Uart module did not have a thorough test suite as it was one of the earliest if not the first thing we developed for the beagle bone black. The testing for this module was done pretty much all throughout our code after uart init. We were able to call all of the print functions to debug the rest of our code like uart_puts(), uart_printf(), print_number() were all extensively used and successfully got arbitrary characters printed through the console. uart_getline() was not put through as rigorous of a test sequence but it was still used extensively in our mvp code being called to retrieve lines received from the RF transmitter which means it was successfully initialised and utilized.

5.3. Process Management

Our Process Management test is very simple. We just want to test creating the process and calling the scheduler to have its context switch out of the process and into another one then back and forth making sure that both processes are executing.

The test code for this looked rather simple here is our process we will be switching in and out of there will be two of them exactly the same

```
{
void test_procx() {
    volatile int count = 0;
    uart_puts("PX running...\n");
    while (1) {
        uart_printf("heartbeat from proc X: %d\n", heartbeat);
        count = 0xFF;
        while (count--) {}
        heartbeat++;
        yield();
    }
}
```

They both print the value of a global counter, wait for a few ticks and increment the global variable then yield for the scheduler to scheduler the next process.

The test code we used to test this roughly amounts to the following but with much more lines :

```
{
void scheduler_test() {
    scheduler_init
    /* Create P1 */
    uart_puts("Creating P1 process...\n");
    proc = process_create(P1);
    ...
    uart_printf("Created process: %s\n", proc->name);
    generic_circular_buffer_push(&processes, (void*)proc);
    generic_circular_buffer_push(&ready_queue, (void*)proc);
    scheduler.num_processes++;

    /* Create P2 */
    uart_puts("Creating P2 process...\n");
    proc = process_create(P2);
    ...
    scheduler_run()
}
```

The output we received was exactly what was expected from our processes having two processes increment counters and yielding to the scheduler through a syscall

```
Scheduler running...
Scheduler: starting proc: P1, pc: a0005e44
P1 running...
P1: heartbeat: 0
P1: Yield
Scheduler running...
Scheduler: process P1 added to ready queue
Scheduler: starting proc: P2, pc: a0005eec
P2 running...
P2: heartbeat: 0
P2: Yield
Scheduler running...
Scheduler: process P2 added to ready queue
Scheduler: starting proc: P1, pc: a0005edc
P1: Return after Yield
P1: heartbeat: 1
P1: Yield
Scheduler running...
Scheduler: process P1 added to ready queue
Scheduler: starting proc: P2, pc: a0005f84
P2: Return after Yield
P2: heartbeat: 1
P2: Yield
Scheduler running...
Scheduler: process P2 added to ready queue
Scheduler: starting proc: P1, pc: a0005edc
P1: Return after Yield
P1: heartbeat: 2
```

5.4. Spinlock

After initialization, I performed the following steps

1. Acquire lock 0 – success
2. Acquire lock 0 – fail
3. Release lock 0
4. Acquire lock 0 - success

Since all locks are different registers, as long as the enum register values are not copied wrong, if one lock succeeds, all other locks will succeed.

5.5. Mailbox

After initialization, I performed the following steps

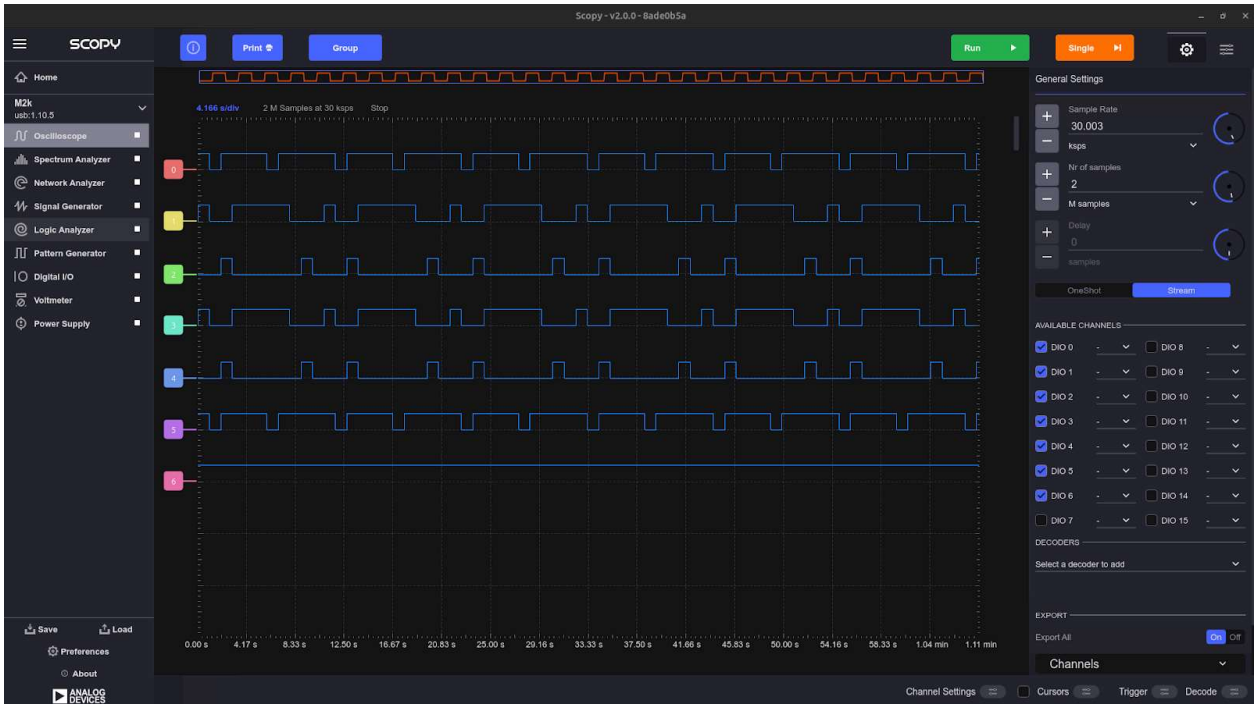
1. While can put messages, put message into box 0
2. Check how many was put into it → 4 messages in total, values in order
    a. 7, 8, 9, 10
3. While can retrieve messages, retrieve from message box 0
4. Check how many was read out → 4 messages read, values in order
    a. 7, 8, 9, 10

This is the exact fifo behavior and it was also only able to queue 4 messages for that one mailbox.

5.6. GPIO

The GPIO pins were first tested after initialization by toggling the debug LEDs and visually verifying that they responded as expected. This basic test confirmed that the GPIO configuration and output functionality were working correctly. Once verified, the same GPIO pins were repurposed for other tasks—most notably, controlling the motor driver. This secondary use case acted as a more practical and sustained test of the GPIO system. While setting up motor control, we used a logic analyzer to confirm that the GPIO pins were being set and cleared in the correct sequence required to drive the motors properly. This provided confidence that both our software control logic and electrical connections were functioning as intended.

Sample logic analyzer output:



5.7. MMC Driver

After initialization, I performed the following steps to make sure the MMC controller's mmc_read_sector() is working properly:

1. Read physical sector 0 into a buffer by manually calling mmc_read_sector with 0 as an argument and an empty buffer.
2. Verify the last 2 bytes (510, 511) are the magic numbers (0x55, 0xAA). This verifies:
    a. The controller is talking to the card properly.
    b. The mmc_read_sector() function is copying data into RAM properly.
    c. The card responds to command 17 properly.
    d. Endian handling and buffer alignment is correct.

    We ran this test and it passed verifying that mmc_read_sector() was working as expected.

    ==== MMC TEST BEGIN ====
    sending CMD17
    waiting for command to complete
    command complete STAT: 100001
    after CC flag cleared STAT: 0
    buf[510] = 0x55 buf[511] = 0xAA
    Tick: 2
    Tick: 3
    ==== MMC TEST COMPLETE ====

5.8. I2C

See Hdmi testing

5.9. MMU

After Initialization is performed we verify a couple things to make sure the MMU is working properly:

1) Call MMU_init() and Verify that Uart prints "Mapped all hardware section entries" and "Loaded L1 tables located at 0xXXXXXXXX into TTBR0" meaning that the tables were successfully set up and loaded in

2) Check execution permission. Query the L1 entry at a valid known space in DRAM such as 0x9fe00000 and observe that the valid section mappings with flags is printed denoting that the MMU sees the sections correctly with the correct permissions aka the same ones we set should be read back.

3) Call MMU_enable() and observe a uart_print of "MMU Enabled!" this means all of the caches were invalidated and flushed properly.

4) Observe that all hardware sections are mapped such as the hardware sections 0x403, 0x44C, 0x480 etc. We can try to access them and they will won't cause the MMU to error this is verified through rerunning the other tests after the MMU is initialised

When performing the initial creation of the driver we ran a manual test of the mmu after running MMU_init and MMU_enable:

```
{
        // Step 1: Manually map a test virtual address to known physical location
        uint32_t* l1_table = (uint32_t*)MEM_BOOT_PAGE_TABLE_BASE;
        uint32_t test_vaddr = 0xA0000000;
        uint32_t test_paddr = 0x80000000;
        uint32_t flags = L1_ACCESS_RW_NO | L1_CACHEABLE | L1_SHAREABLE;

        uart_puts("Manually mapping 0xA0000000 -> 0x80000000\n");
        MMU_map_section(l1_table, test_vaddr, test_paddr, flags);
        flush_tlb_entry(test_vaddr);

        // Step 2: Log mapping of test address
        log_vaddr_mappings((void*)test_vaddr);

       // Step 3: Attempt test access to verify mapping
        volatile uint32_t* test_ptr = (uint32_t*)test_vaddr;
        *test_ptr = 0xDEADBEEF;
        if (*test_ptr == 0xDEADBEEF) {
            uart_puts("Test access PASSED: value retained at mapped virtual address\n");
        } else {
            uart_puts("Test access FAILED: could not read back value\n");
        }

}
```

This test ran and passed manually verifying the logs as well as verifying the test pointer.

```
==== MMU TEST BEGIN ====
Mapped all hardware section entries
Loaded L1 tables located at 0x90000000 into TTBR0
Check: You should see 'Mapped all hardware section entries'
Check: You should see 'Loaded L1 tables located at 0x90000000  into TTBR0
Reading L1 entry for 0x9FE00000:
VADDR: 0x9fe00000
L1PageTableEntry { base: 0x9fe00000, B: 1, C: 1, AP: 0 1, TEX: 0, Domain: 0, nG: 0, S: 1, XN: 0, NS:
0, type: Section }
MMU Enabled!
Manually mapping 0xA0000000 -> 0x80000000
VADDR: 0xa0000000
L1PageTableEntry { base: 0x80000000, B: 1, C: 1, AP: 0 1, TEX: 0, Domain: 0, nG: 0, S: 1, XN: 0, NS:
0, type: Section }
```

Test access PASSED: value retained at mapped virtual address
==== MMU TEST COMPLETE ====

This test however is not included as it requires for our os to not be present.

5.10. Tda19988 (Hdmi framer)

After initializing I2C0 :

1. Call hdmi_framer_probe() → Prints TDA19988 to the uart console
    a. For this to print, the tda19988's cec chip must have
        i. Acknowledged the I2C write and read request or I2C will either hang and block because the cec chip is turned off or return -1 because the slave device did not acknowledge the request
        ii. responded with the value 0x0301

Since the print statement happened, I know that I2C successfully and correctly handled I2C communication protocol and the framer was able to ping the cec chip with the correct commands at the correct location

All I2C lines go through the same initialization process, since I2C0 worked, I2C1 and I2C2 would have worked. But then again, our implementation only really needed I2C0.

5.11. LCD Controller

After initializing Frame buffer, I2C, and hdmi framer as noted in the API :

1. Called LCD_init()
2. The hdmi monitor responded by turning on its indication lights
    a. For around 15 - 20 seconds before going back to sleep
    b. This means we have a pixel clock from the LCD controller going to the monitor through the hdmi framer
3. Repeatedly calling ProbeIntrCode() prints out IRQ (2) and (5)
    a. This demonstrates that the LCD controller is active and attempting to transfer data through the DMA engine.
4. Exact reason for those irq status are unclear yet

6. The good

6.1. MMU :

Although this ended up being one of our biggest hindrances after we implemented it, the fact that we successfully implemented an MMU at all is quite a massive achievement. Our MMU successfully initialises the memory we have access to and assumes control over it. The API implemented for the MMU is fully functional including debugging, various disables and enables for certain features and includes an easy to understand function to set permissions on each individual page of the RAM for any of the many modes. A great achievement of our MMU was our one to one mapping of all of our hardware registers that access all our peripherals allowing the kernel full access to the registers while blocking any user

process from reading or writing to them. Perhaps if we had more time and worked with it more the MMU may have become a tool that could help us rather than hinder our development.

6.2. Meeting the MVP :

We successfully implemented a drivable car as described in the MVP, wireless controlled. Although compared to other groups, our OS did not have any of the fancy UDP drivers or robust file system, it achieves what we set out to do - driving a car. Our OS is built for one task that it performed very well on that task which we set out to do. We faced a lot of hardware (the car) and software challenges which we believe is unique to our own group as our OS diverges quite a bit from other OSes in terms of features. We are not claiming that our OS is better than the other OSes, but rather it meets the MVP fully and some more. Multitasking, file systems, I/O, are all important for a user oriented operating system, but for the sole purpose of driving a wireless car they are not necessary.

7. The bad:

As we can all see the final implementation of the project was incomplete, lacking some key aspects that any operating system should have and lacking any formal organization. This was in large part due to poor communication and poor planning. This can be attributed to many issues such as; an incomplete objective, work duplication, poor communication, overextended ambition and a general lack of direction.

The first we encountered was the lack of a clear objective that was made apparent when we finished the bootloader and did not know what to do. We were not familiarized with the process of coding an operating system and had no clue what our greatest challenges would be. So the work was split up rather unevenly in terms of actual weight towards the project having one person work on the core process and memory, one on the MMU, one on a hdmi driver, and one on the vehicle aspect with the others helping. The designated division of labour resulted in the entire group depending on one person finishing the process so if the person working on the main aspect of the os did not finish the extra nice to haves turn from nice to haves to a gross misunderstanding of project management. It would have been much wiser to first divide up the work of the main section only focusing on the nice to haves later. If we had divided up the main kernel into sections like Memory, MMU, Process design, Scheduler, and context switcher we could have completed the main section much quicker having something to show for our efforts rather than having a bunch of different monolithic undertakings half finished.

The second issue we come to is work duplication. Everyone in the group at the beginning was more interested in the bootloader and not much on the OS itself, so the majority of our effort went into the bootloader. However, since everyone was interested in the bootloader, we had a lot of duplicated sections of code. Separation of tasks at the beginning of the semester was ignored and almost every group member began working on the same issue at hand all at once.

This leads us to a clear lack of communication due to how the group functioned. Most members of the group preferred working on their own and then coming together to combine their implementations. This issue caused the duplication such as 3 different implementations of GPIO initialization being made, and 2 different implementations of interrupt initializations and handling existed. Only 1 of each was selected,

despite a lot of effort from each creator. Even when doing duplicated work, for instance for the interrupt handler, there was no communication and sharing of knowledge. We all read and learned about the ARM interrupt sequence on our own, this is a giant inefficiency.

Leading to our last issue being our overenthusiastic ambition. The MVP we had in mind was a drivable car, we did not necessarily need all of the fancy features that are present in the current implementation of the os. We implemented massive features like an MMU and an hdmi driver. These features have no business being in an operating system coded in 3 (more realistically one and a half) months, especially an operating system meant to drive a car that will have no real user processes. An MMU is meant for a much more complex system to protect memory from the user and facilitate coding. In our case the MMU ended up being the biggest hindrance and probably the number one reason for not finishing on time. We however, insisted on including the features anyway because we were too ambitious with what we wanted to accomplish. We dreamed too big, had way too many feature distractions (feature bloat) without a solid foundation to build from. In the end failed to deliver an OS that we were satisfied with.

8. Learnings:

This project was a very interesting learning opportunity for the members of our group being able to tackle several interesting diverse problems that taught us many different things about the design, operation and scale of many basic operating system and computer hardware features. We were able to learn about the details of an OS first hand. There was also a lot of preliminary research into architecture specific topics we went into such as how the armv7 instruction set functions and how an am355x utilises those to do specific tasks on the beagle bone black. We learned about how a processor interfaces with its hardware and just how difficult those things are to set up.

Outside of architecture specific knowledge we learned about some truths of os and probably general development. A lot of problems, especially those at our level have all been solved, published and documented in some capacity. Reverse engineering thos solutions is not just an option it's sometimes a necessity for when you cannot get a rigid module such as the DRAM or the MMU to work. It is much easier to reverse engineer than it is to try to initialise from the ground up. Reverse engineering even if you don't use the initial reverse engineered code you synthesise will teach you a lot about how a specific thing works and allow your second implementation of that module to be implemented quicker and be far more effective then if you had just written it by yourself and cut corners or wasted a lot of time.

Moving onto some lessons we should have learned already and are retracing again on this project is a need to test early and test often. If we had made a robust testing suite or testing branch of our code it would've been much easier to find and isolate bugs rather than blindly print everytime our code didnt work without so much as a clue as to where the problem may actually lie. We may have also benefited from retesting things often as occasionally we would make many changes but as we make a change we break some other mechanism and would have to go and fix it again a proper testing suite would have easily simplified and expedited this process.

More from the playbook of things we should already know is the need to document as we go. It would have been much simpler for us to communicate and integrate if after we wrote a module or some piece of code we sat down for five minutes and word vomited onto a page about what we wrote and then organised them into bullet points into a central document. This would have also kept us up with what each group member was doing, reduced code duplication and allowed us to move faster potentially fast enough to finish the project on time.

There are many other lessons we learned from this project but we will go to another big one and that is premature over engineering. We went over this in the section discussing things that did not go well but we had a mvp that we should have stuck to before trying to do more complicated things like bringing in an MMU. The time we spent trying to engineer processes around the MMU alone could have been repurposed to implement full context switching and scheduling as well as a shell and user space processes without MMU protection. This lesson can serve as a warning for when we work on a project in the future or when we go to work for a company. The warning of course being to focus on the MVP and make sure its done early and without any cut corners once you have an MVP finished any extra time you have left over you can do whatever you wish with like in our case an MMU or LCD screen.

One of the biggest challenges we faced was the lack of continuous integration throughout the project. While we successfully implemented several critical components—UART drivers, timer functionality, GPIO control, interrupt handling, syscalls, basic scheduling, and more—these were often developed and tested in isolation. As a result, we ended up with a number of functional modules that could, in theory, form the backbone of a solid operating system, but were never fully integrated into a cohesive whole. This disjointedness made final testing and debugging more difficult, and limited our ability to demonstrate a fully working system. In hindsight, integrating modules incrementally and testing them as a system early on would have exposed integration issues sooner and helped align development across the team.

We significantly underestimated the importance of project management. Early on, we lacked clear milestones, division of responsibilities, and regular check-ins to align on priorities. This led to uneven progress across subsystems, duplicated efforts in some areas, and critical features being postponed until too late in the timeline. We often worked on components we personally found interesting, rather than prioritizing what was needed to meet project goals. This contributed to the final product being more of a collection of working parts rather than a fully polished system.

Overall, we were able to deliver an BBB OS that achieves our MVP. There were a few bumps along the way and many hard (technical) and soft (personal) lessons learned along the way. We had a lot of individual systems implemented, but not enough time to integrate the entire system together. Although we were not satisfied with the final outcome of the OS, we are still proud to present the features that we had.